Framework de desarrollo seguro basado en métodos computacionales guiados por conocimiento

Informe del sprint 2

Conocimiento, OPA y Telegram: walking skeleton con los tres Security Gates

19 al 30 de octubre de 2026

Marcelo Acuña Gamboa — Participante 1: motor e IA

Abner Arboleda — Participante 2: plataforma y seguridad

Universidad de las Fuerzas Armadas – ESPE • Ingeniería de Software

Versión 1.0 • 1 de noviembre de 2026

Índice

1. Resumen del sprint **2**
- 1.1. Objetivo e hito . . . 2
- 1.2. Indicadores . . . 3

2. Trabajo realizado **4**
- 2.1. Cumplimiento del cronograma . . . 4
- 2.2. Motor e IA . . . 4
  - 2.2.1. Base de conocimiento v1 . . . 4
  - 2.2.2. Carga, embeddings y GraphRAG . . . 5
  - 2.2.3. Políticas OPA de los Security Gates . . . 5
  - 2.2.4. Decisiones de los gates en el motor y en Neo4j . . . 6
  - 2.2.5. Benchmark de bases de datos . . . 6
- 2.3. Plataforma y seguridad . . . 7
  - 2.3.1. Avisos en Telegram . . . 7
  - 2.3.2. Security Gate 2: integración . . . 7
  - 2.3.3. Security Gate 3: sistema en marcha . . . 7

3. Revisión de la base de conocimiento v1 **8**
- 3.1. Resultado de las comprobaciones . . . 8
- 3.2. Hallazgos y correcciones . . . 8
- 3.3. Brechas que quedan para la versión 2 . . . 8

4. Estado del pipeline al cierre **10**
- 4.1. Walking skeleton con los tres gates . . . 10
- 4.2. Fases del pipeline . . . 10
- 4.3. Alertas abiertas en GitHub Security . . . 10

5. Incidencias y lecciones aprendidas **12**

6. Seguridad del propio framework **13**

7. Planificación del sprint 3 **14**
- 7.1. Objetivo . . . 14
- 7.2. Dependencias y preparación . . . 14

A. Evidencias **16**

1. Resumen del sprint

1.1 Objetivo e hito

El sprint 2 completa el walking skeleton del principio P3 (*primero de punta a punta, después inteligente*). Su objetivo era construir la base de conocimiento v1 en Neo4j con GraphRAG, escribir las políticas OPA de los tres Security Gates, levantar las versiones iniciales de los Gates 2 y 3 sobre el servicio de referencia, avisar del progreso por Telegram y comparar Neo4j con PostgreSQL + pgvector.

Hito de control del 30 de octubre de 2026

Walking skeleton completo (tres gates) con avisos en Telegram. Es el primer hito importante del proyecto: no se empieza el sprint 3 sin haberlo cumplido.

Resultado: hito cumplido

El 30/10/2026 la ejecución 41135849778 de GitHub Actions sobre `dev` recorrió las fases 3 a 11 con los tres gates encadenados (#174) y Telegram informó de cada fase. El 01/11/2026 se integraron las decisiones de OPA (#176): en la ejecución 41341510212 los tres gates fueron aprobados por las políticas Rego, con 4 de 4, 5 de 5 y 5 de 5 criterios cumplidos. Cada decisión quedó registrada en Neo4j como un nodo `ResultadoGate`, junto con las reglas que la justifican. Se puede pasar al sprint 3.

1.2 Indicadores

Tabla 1.1: Indicadores del sprint 2, medidos en la ejecución 41341510212 sobre `dev` y en local al cierre.

Indicador	**Valor**
Días y horas del sprint	10 días, 80 horas de equipo (40 por integrante)
Pull requests fusionados en `dev`	21 (#157 a #177; 10 de Marcelo y 11 de Abner), todos con revisión del otro integrante, más el de este informe
Duración del pipeline completo	10 min 50 s (Gate 1: 40 s; Gate 2: 7 s; Gate 3 con DAST: 5 min 20 s)
Decisión OPA de los gates	3 de 3 aprobados: Gate 1 con 4/4 criterios, Gate 2 con 5/5, Gate 3 con 5/5
Base de conocimiento v1	6 patrones, 6 directrices, 27 controles ASVS, 47 CWE y 10 riesgos OWASP API; 113 relaciones
Fragmentos para GraphRAG	96 con embeddings de 384 dimensiones en el índice vectorial `fragmento_embedding`
Pruebas de políticas Rego	64 en verde (36 de los gates con 100 % de cobertura y 28 de integración)
Pruebas del motor (pytest)	248 en verde, incluidas las de integración con Neo4j; 92 % de cobertura de `orchestrator/`
Pruebas del servicio (Jest)	42 en verde, 94,7 % de cobertura de sentencias
DAST sobre el servicio en marcha	ZAP: 115 reglas sin fallo y 4 avisos de severidad baja; Nuclei: 0 coincidencias
Imagen y SBOM	Trivy image: 0 vulnerabilidades altas o críticas; SBOM CycloneDX 1.7 con 1866 componentes
Alertas abiertas en GitHub Security	47, todas de severidad advertencia o baja (ninguna bloquea un gate)

2. Trabajo realizado

2.1 Cumplimiento del cronograma

La tabla compara el cronograma diario del plan con lo entregado. La columna PR indica el pull request con el que cada tarea entró en `dev`.

Tabla 2.1: Tareas del sprint 2 por integrante y pull request asociado.

Fecha	Marcelo (motor e IA)	Abner (plataforma y seguridad)	PR Marcelo	PR Abner
Sprint 2: Conocimiento, OPA y Telegram				
19/10 lun	Patrones de microservicios y directrices Secure by Design en YAML, con validador.	Notificador de Telegram: bot y mensaje de progreso editable.	#157, #158	#157
20/10 mar	Controles OWASP ASVS y catálogos CWE y OWASP API Top 10.	Avisos en cada fase del workflow y aviso de fallo.	#160	#159
21/10 mié	Cargador YAML → Neo4j (nodos y relaciones).	Gate 2 inicial: Spectral con reglas OWASP API.	#162	#161
22/10 jue	Fragmentos con embeddings e índice vectorial.	Conftest con políticas Rego para Docker Compose.	#165	#163
23/10 vie	Consulta GraphRAG (vectores + relaciones).	Realm de Keycloak y guard JWT en el servicio.	#166	#164
26/10 lun	Políticas Rego de los Gates 1 y 2 con `opa test`.	Gate 3 inicial: Trivy image y SBOM con Syft.	#168	#171
27/10 mar	Política Rego del Gate 3 con justificación.	ZAP y Nuclei autenticados con límite de tiempo.	#169	#172
28/10 mié	Benchmark Neo4j frente a PostgreSQL + pgvector.	Conversor de resultados de ZAP a SARIF.	#170	#173
29/10 jue	Decisiones OPA de los tres gates en el motor y en Neo4j.	Gates 1–3 encadenados con avisos en Telegram.	#176	#174
30/10 vie	Revisión de la base de conocimiento v1 e informe del sprint 2.	Demostración del skeleton completo y planificación del sprint 3.	este PR	#177

Las tareas de plataforma entraron en su fecha o con un día de diferencia. Las de políticas del motor se retrasaron: #168 se fusionó el 29/10, y #169, #170 y #176 el 01/11. La causa fue la cadena de ramas: la del Gate 3 se creó sobre la de los Gates 1 y 2 antes de que esta se fusionara, y al fusionarla con *squash* el PR del Gate 3 quedó en conflicto (ver el capítulo 5). El retraso no afectó al hito, que dependía del encadenamiento de los gates (#174).

2.2 Motor e IA

2.2.1 Base de conocimiento v1

La base de conocimiento (principio P5) está escrita en YAML dentro de `knowledge_base/`, se revisa por pull request y un validador comprueba su estructura y sus referencias cruzadas

antes de cargarla. Cada elemento cita su fuente: OWASP ASVS 4.0.3, MITRE CWE y OWASP API Security Top 10 (2023).

Tabla 2.2: Contenido de la base de conocimiento v1 tras la revisión del 30/10.

Tipo	**Cantidad**	**Relaciones en el grafo**
Patrones de microservicios	6	`MITIGA` CWE; los controles `SE_IMPLEMENTA_CON` el patrón
Directrices Secure by Design	6	`MITIGA` CWE; los controles `SE_IMPLEMENTA_CON` la directriz
Controles OWASP ASVS 4.0.3	27	`CUBRE` CWE y `PREVIENE` riesgos de API
Debilidades CWE	47	Destino de las relaciones anteriores y de los hallazgos
Riesgos OWASP API Top 10	10	`INCLUYE` CWE
Fragmentos	96	`DESCRIBE` el elemento del que salen; embedding de 384 dimensiones

2.2.2 Carga, embeddings y GraphRAG

- **Cargador** (#162): `framework conocimiento cargar` valida la base y la escribe en Neo4j con `MERGE`, de modo que cargarla dos veces no duplica nada.
- **Fragmentos e índice vectorial** (#165): cada elemento se convierte en un fragmento de texto con su embedding (`paraphrase-multilingual-MiniLM-L12-v2`, 384 dimensiones, multilingüe) y se indexa con un índice HNSW por similitud de coseno. Los fragmentos llevan la versión del contenido y del modelo, y los de versiones anteriores se retiran.
- **GraphRAG** (#166): `framework conocimiento consultar` busca los fragmentos más parecidos a la pregunta y, desde ellos, recorre el grafo para traer el conocimiento conectado: CWE, patrones, directrices, controles y riesgos de API. Así el LLM recibe contexto verificable y trazable, no solo texto parecido.

2.2.3 Políticas OPA de los Security Gates

Las políticas de `policies/gates/` deciden si cada gate se aprueba (principio P1: el LLM propone, las reglas deciden). Son deterministas, tienen pruebas con `opa test` y fallan de forma segura: si falta un dato, el gate no se aprueba. Cada decisión devuelve la lista completa de criterios con `cumple` y su descripción, de modo que también una aprobación queda justificada regla por regla.

Tabla 2.3: Criterios de las políticas Rego de cada gate.

Gate	Criterios (todos deben cumplirse)
1	Sin hallazgos críticos o altos; sin secretos (Gitleaks); el código compila; ninguna herramienta caída.
2	Contrato OpenAPI generado; sin errores de Spectral; sin `deny` de Conftest en el Docker Compose; sin otros hallazgos críticos o altos; ninguna herramienta caída.
3	Sin vulnerabilidades críticas o altas en el DAST (ZAP, Nuclei); ni en la imagen (Trivy image); SBOM generado; todas las pruebas de la fase 9 en verde; ninguna herramienta caída.

2.2.4 Decisiones de los gates en el motor y en Neo4j

`framework gates decidir` (#176) arma la entrada de OPA a partir de los artefactos del pipeline (SARIF, contrato OpenAPI, SBOM y resultado de Jest), pide la decisión, la guarda en `salida/gates/` y la registra en el grafo. Los tres gates de un mismo pipeline comparten una ejecución:

```
(:Ejecucion)-[:TIENE_RESULTADO]->(:ResultadoGate)-[:BASADA_EN {cumple, detalle}]->(:Regla)
                (:ResultadoGate)-[:BLOQUEADO_POR {regla}]->(:Hallazgo)
```

En el workflow, un paso *Decisión OPA* sigue a cada gate y detiene la cadena si OPA rechaza. Un pipeline de ejemplo en `tests/datos/gates/` prueba los tres resultados posibles: el Gate 1 se rechaza por un secreto y hallazgos altos, el Gate 2 se aprueba con una advertencia y el Gate 3 se rechaza por una inyección SQL que encuentra ZAP.

2.2.5 Benchmark de bases de datos

La prueba comparativa (#170, `evaluation/benchmark_bd/`) cargó los mismos datos en Neo4j y en PostgreSQL con pgvector y midió 300 veces tres consultas típicas del motor.

Tabla 2.4: Latencia media vista desde el motor (ms), 28/10/2026.

Consulta	Neo4j	PostgreSQL	Resultados iguales
Trazabilidad a 3 saltos (riesgo → CWE → control)	10,2	1,3	10 de 10 riesgos
Controles que protegen frente a una CWE	6,1	1,8	42 de 42 CWE
5 fragmentos más parecidos (vectorial)	7,9	1,5	88 % del top 5

Decisión: se mantiene Neo4j

PostgreSQL fue entre 3 y 8 veces más rápido, pero las dos bases responden en milisegundos y la latencia del pipeline la dominan las herramientas y el LLM (segundos o minutos). Neo4j se mantiene porque el modelo del framework es un grafo de trazabilidad (requisito → decisión → hallazgo → regla) que se consulta con recorridos de longitud variable, y porque reúne el grafo y el índice vectorial en un solo servicio. La comparación queda como anexo para la defensa de la decisión.

2.3 Plataforma y seguridad

2.3.1 Avisos en Telegram

El notificador (#157, #159, #174) mantiene un único mensaje de progreso que se edita en cada fase y envía un mensaje nuevo al terminar, para que Telegram avise del resultado. Si la cadena se detiene, indica en qué gate se detuvo, marca las fases no ejecutadas y también cubre la cancelación. El token del bot nunca llega a los comandos que ejecuta el workflow.

2.3.2 Security Gate 2: integración

- **Spectral** (#161): revisa el contrato OpenAPI del servicio con el conjunto de reglas OWASP API Security y 4 reglas propias (prefijo `/api`, sin secretos en la query, sin campos sensibles en las respuestas y versión semántica), que se autoprueban en cada ejecución con un contrato correcto y otro incorrecto.
- **Conftest** (#163): 11 reglas `deny` y 4 `warn` en Rego sobre el Docker Compose. Bloquean, entre otros, contenedores privilegiados, red del host, puertos abiertos a toda la red, imágenes `latest`, valores escritos en las variables de entorno y bases de datos compartidas entre servicios (patrón *Database per Service*).
- **Keycloak** (#164): realm `framework` y guard JWT en el servicio de referencia; las rutas responden 401 sin token y 200 con un token válido.

2.3.3 Security Gate 3: sistema en marcha

- **Imagen y SBOM** (#171): Trivy image sobre una imagen final *distroless* y SBOM en CycloneDX generado con Syft y validado.
- **DAST** (#172): ZAP con la política completa y Nuclei atacan el servicio, con un token de Keycloak, en un entorno desechable (`docker-compose.dast.yml`) y con límite de tiempo. ZAP encontró un error 500 al enviar un byte nulo en el nombre; se corrigió en la validación del DTO, con pruebas.
- **Conversor ZAP → SARIF** (#173): lleva los resultados de ZAP al formato común, a la pestaña Security y a la decisión de OPA.

3. Revisión de la base de conocimiento v1

La revisión del 30/10 comprobó la base en tres niveles: la estructura de los YAML con el validador, la coherencia del grafo cargado con diez consultas Cypher de solo lectura y la calidad de las respuestas con consultas GraphRAG de prueba.

3.1 Resultado de las comprobaciones

Tabla 3.1: Comprobaciones de la base cargada en Neo4j, antes y después de las correcciones.

Comprobación	**Antes**	**Después**
Validador de los YAML (estructura y referencias)	válida	válida
Nodos de conocimiento sin ninguna relación	0	0
Elementos sin fragmento para GraphRAG	3 CWE	0
Fragmentos sin embedding o con otra dimensión	0	0
CWE citadas por hallazgos que no están en el catálogo	3	0
CWE de las reglas Semgrep propias fuera del catálogo	3 de 8	0 de 8
CWE con al menos una mitigación (control, directriz o patrón)	33 de 42	38 de 47
Riesgos OWASP API con al menos un control	9 de 10	9 de 10

3.2 Hallazgos y correcciones

- **Reglas propias sin respaldo en el catálogo.** Las reglas Semgrep `tls-verificacion-desactivada`, `hash-criptografico-debil` y `jwt-ignora-expiracion` citan CWE-295, CWE-327 y CWE-613, que no existían en el catálogo. Sus hallazgos quedaban en el grafo sin descripción ni controles. Se añadieron esas tres CWE y los controles ASVS 4.0.3 que las cubren según el CSV oficial: V9.2.1, V6.2.2 y V3.3.1. Una prueba nueva (`tests/test_catalogos.py`) falla si una regla propia cita una CWE que no está en el catálogo.
- **CWE que solo llegaban por los hallazgos.** Semgrep reportó CWE-916 (hash de contraseñas débil) y ZAP reportó CWE-693 (falta un mecanismo de protección, como la cabecera CSP). Se añadieron al catálogo, y CWE-916 quedó cubierta por el control V2.4.1.
- **Asignación masiva sin control.** CWE-915, que detecta la regla `nestjs-validationpipe-sin-whitelist`, no tenía ningún control. Se añadió ASVS V5.1.2.
- **Archivo sobrante.** Se eliminó `knowledge_base/patrones/.gitkeep`, que ya no hacía falta.

Tras las correcciones, la consulta GraphRAG *"guardar contraseñas con hash MD5"* devuelve CWE-916 y el control V2.4.1, que antes no existían en la base.

3.3 Brechas que quedan para la versión 2

- Nueve CWE no tienen ninguna mitigación: CWE-204, 213, 307, 444, 693, 799, 918, 942 y 1059. CWE-942 (CORS permisivo) la detecta la regla propia `nestjs-cors-abierto`, pero

ASVS 4.0.3 asocia su control de CORS (V14.5.3) a CWE-346, así que no se inventó el vínculo.

- El riesgo API9:2023 (gestión de inventario de API) no tiene ningún control.
- API6:2023 no tiene CWE porque OWASP no publica ninguna para ese riesgo; es correcto y se mantiene así.
- Las reglas Semgrep propias todavía no son nodos del grafo. El sprint 1 lo dejó previsto y conviene hacerlo antes del sprint 4, cuando el Gate 1 se aplique a código generado.
- El benchmark del 28/10 se midió con la base anterior a esta revisión (86 elementos). Los cambios no alteran sus conclusiones.

4. Estado del pipeline al cierre

4.1 Walking skeleton con los tres gates

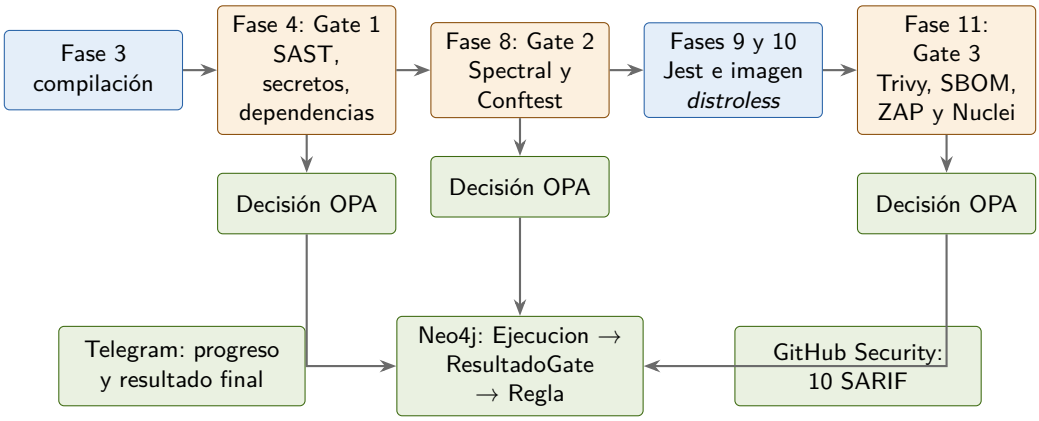


Figura 4.1: Recorrido del walking skeleton en cada ejecución de GitHub Actions. Si una decisión de OPA rechaza un gate, la cadena se detiene y Telegram indica dónde.

4.2 Fases del pipeline

Tabla 4.1: Estado de cada fase del workflow al cierre del sprint 2 (ejecución 41341510212).

Fase	Nombre	Estado	Detalle
1–2	Requisitos y arquitectura	Pendiente	Sprint 3.
3	Generación y compilación	Versión inicial	Compila el servicio de referencia (57 s).
4	Security Gate 1	Con decisión OPA	8 comprobaciones, 5 SARIF, registro en Neo4j (40 s).
5–6	Refinamiento iterativo	Pendiente	Sprint 5.
7	Integración	Parcial	Keycloak con guard JWT; Traefik y RabbitMQ en el sprint 5.
8	Security Gate 2	Con decisión OPA	Spectral y Conftest sobre el servicio de referencia (7 s).
9	Pruebas automatizadas	Versión inicial	42 pruebas Jest con cobertura (4 s).
10	Contenerización	Versión inicial	Imagen *distroless* sin root (1 min 14 s).
11	DAST y Security Gate 3	Con decisión OPA	Trivy image, Syft, ZAP y Nuclei autenticados (5 min 20 s).

4.3 Alertas abiertas en GitHub Security

Las 47 alertas abiertas en `dev` son de severidad advertencia o baja, por lo que las políticas las tratan como advertencias y no bloquean ningún gate. Quedan como deuda técnica del servicio

de referencia:

Tabla 4.2: Alertas abiertas en `dev` al 01/11/2026.

Herramienta	**Alertas**	**Reglas principales**
Spectral	25	Falta limitación de peticiones y respuesta 429 (API4); falta definir las respuestas 500 y de validación (API8)
Conftest	14	Avisos sobre los Compose de desarrollo y DAST: sin *healthcheck*, sin `no-new-privileges`, imágenes sin digest y Keycloak en modo `start-dev`
ZAP	8	Cabecera `X-Powered-By`, `X-Content-Type-Options` y `Cross-Origin-Resource-Policy`

5. Incidencias y lecciones aprendidas

Como en el sprint 1, ninguna incidencia se resolvió relajando un gate.

Tabla 5.1: Incidencias del sprint 2 y su corrección.

Incidencia	Causa	Corrección	PR
Gate 1 en rojo el 24/10	Vulnerabilidades altas publicadas en `body-parser`, `multer` (servicio) y `pillow` (motor).	Actualización a las versiones corregidas en `package-lock.json` y `uv.lock`.	#167
Gate 1 en rojo el 01/11	Cuatro CVE altas nuevas en `pyasn1` 0.6.4, dependencia indirecta del cliente de Gemini.	`pyasn1` 0.6.5; Trivy sin hallazgos sobre el nuevo `uv.lock`.	#175
Error 500 hallado por ZAP	Un byte nulo en el nombre llegaba hasta PostgreSQL, que lo rechaza.	El DTO rechaza caracteres de control; pruebas con byte nulo, salto de línea y DEL.	#172
PR del Gate 3 en conflicto	La rama se creó sobre la de los Gates 1 y 2 antes de fusionarla; el *squash* cambió el identificador de ese commit.	Se reaplicó solo el commit propio sobre `dev` (`git rebase --onto`).	#169
Gate 3 rechazado con un escaneo limpio	Nuclei no crea su SARIF cuando no encuentra nada y OPA lo trata como herramienta caída.	`gate3.sh` deja un SARIF sin resultados cuando el escaneo termina limpio.	#176
Cada gate creaba su propia ejecución	Sin fase 4 previa, no quedaba registro de la ejecución creada.	El primer gate guarda su ejecución en `salida/gates/ejecucion.json` y los demás la reutilizan.	#176
Docker Desktop no arranca (local)	El *secrets engine* no podía abrir su socket en `%LOCALAPPDATA%`.	Se renombró la carpeta del *secrets engine* y se reinició Docker, sin restablecer los datos de fábrica.	–
Versión de Python distinta en local	Sin `.python-version`, uv eligió CPython 3.15 en una copia nueva del repositorio.	Se fijará 3.12 con `.python-version` en un PR propio.	pendiente

Lecciones principales

- **Las dependencias cambian sin tocar el código.** Dos veces el Gate 1 pasó a rojo por CVE publicadas durante el sprint. Es el comportamiento esperado del gate, y obliga a reservar tiempo para actualizar dependencias en cada sprint.
- **Ramas cortas y desde `dev`.** Encadenar ramas sin fusionar, con fusión por *squash*, produce conflictos artificiales. Cada tarea debe partir de `dev` actualizado y fusionarse antes de empezar la siguiente que dependa de ella.
- **Ausencia de dato frente a dato vacío.** Con el fallo seguro de OPA, una herramienta que no deja evidencia de un escaneo limpio provoca un rechazo. Cada herramienta debe dejar siempre su SARIF, aunque esté vacío.

6. Seguridad del propio framework

Controles del capítulo 10 del plan añadidos en el sprint 2, además de los del sprint 1:

Tabla 6.1: Controles del capítulo 10 implementados en el sprint 2.

Riesgo	**Control activo**
Decisiones de seguridad delegadas al LLM	Las aprueba o rechaza OPA con políticas versionadas y probadas (64 pruebas Rego en CI); el LLM no interviene.
Decisiones sin trazabilidad	Cada decisión queda en Neo4j con su ejecución, sus criterios y los hallazgos que la bloquean.
Manipulación del conocimiento	La base es YAML revisado por PR; el validador y las pruebas comprueban sus referencias, incluidas las CWE de las reglas propias.
Fragmentos desactualizados en GraphRAG	Cada fragmento lleva la versión del contenido y del modelo; los de versiones anteriores se retiran al recargar.
Credenciales de Telegram	Token y chat en GitHub Secrets; los comandos que ejecuta el workflow no los reciben.
DAST sobre servicios reales	ZAP y Nuclei atacan un entorno desechable en una red aislada, con límite de tiempo.
Imagen del servicio	Imagen final *distroless*, sin shell ni gestor de paquetes, con SBOM verificable.

7. Planificación del sprint 3

7.1 Objetivo

Sprint 3: fases 1 y 2 (4 al 13 de noviembre de 2026)

Del documento de requisitos al modelo estructurado, la arquitectura validada por OPA, los ADR y el diagrama C4. Dataset de evaluación y primera versión del dashboard. Son 8 días y 64 horas de equipo. El siguiente hito de control es el del 27 de noviembre: el framework genera microservicios desde un documento de requisitos y les aplica el Security Gate 1.

Tabla 7.1: Cronograma del sprint 3 por integrante.

Fecha	Marcelo (motor e IA)	Abner (plataforma y seguridad)
04/11 mié	Docling: extracción de PDF, DOCX y TXT.	Dataset: documento de requisitos 1 (tienda en línea).
05/11 jue	Normalización del texto y esquema Pydantic `Requisito`.	Documentos 2 y 3 (gestión académica y citas médicas).
06/11 vie	Clasificación RF/RNF y etiquetas de seguridad con LLM.	Dashboard React + Vite: estructura y carga del documento.
09/11 lun	Requisitos guardados como nodos en Neo4j.	Dashboard: progreso por fase con Server-Sent Events.
10/11 mar	Propuesta de arquitectura con LLM (contextos y servicios).	API del motor: endpoints de ejecución y estado.
11/11 mié	OPA: reglas arquitectónicas e inferencia de patrones.	Generación de ADR en Markdown.
12/11 jue	Ciclo LLM OPA (máximo 3 intentos) y nodos `Decision`.	Diagrama C4 en Mermaid.
13/11 vie	Ambos: fases 1–2 con los 3 documentos, revisión de ADR y diagramas, demostración e informe del sprint 3.	

7.2 Dependencias y preparación

La planificación detallada, con las issues de cada tarea, los riesgos y los pendientes del sprint 2, está en `docs/sprints/sprint3-planificacion.md` (#177).

- **GraphRAG en la fase 2.** La propuesta de arquitectura (10/11) es el primer uso real de GraphRAG: el LLM recibirá los patrones, directrices y controles conectados a los requisitos. Las brechas del capítulo 3 deben cerrarse antes o quedar documentadas en el ADR.
- **OPA para la arquitectura (11/11)** reutiliza el patrón de los gates: entrada armada por el motor, criterios con `cumple` y registro en Neo4j (nodos `Decision` enlazados a `Regla`, como `ResultadoGate`).
- **Dataset (04–05/11).** Los tres documentos de requisitos son también la entrada de los experimentos DSR, así que deben quedar versionados y sin cambios después del sprint 3.
- **Dashboard y API (06–10/11).** La API del motor debe exponer el estado de las fases antes de que el dashboard lo consuma; conviene acordar el formato de los eventos el 06/11.

- **Deuda heredada.** Fijar Python 3.12 con `.python-version`, revisar las 47 alertas de severidad advertencia o baja y añadir las reglas Semgrep propias al grafo.

A. Evidencias

Tabla A.1: Dónde comprobar cada resultado del sprint.

Evidencia	**Ubicación**
Ejecución del hito (30/10)	`https://github.com/PracticasESPE/Framework_Desarrollo_Seguro/actions/runs/41135849778`
Ejecución con decisiones OPA	`https://github.com/PracticasESPE/Framework_Desarrollo_Seguro/actions/runs/41341510212`
Pestaña Security (SARIF)	`https://github.com/PracticasESPE/Framework_Desarrollo_Seguro/security/code-scanning`
Base de conocimiento	`knowledge_base/`
Cargador, embeddings y GraphRAG	`orchestrator/knowledge/`
Políticas de los gates	`policies/gates/` (con su README)
Decisiones de los gates	`orchestrator/gates/decision.py` y `tests/datos/gates/`
Benchmark de bases de datos	`evaluation/benchmark_bd/resultados.md`
Gate 2	`scripts/gate2.sh`, `spectral/`, `policies/integracion/`
Gate 3 y DAST	`scripts/gate3.sh`, `dast/`, `docker-compose.dast.yml`
Keycloak	`keycloak/realm-framework.json`
Notificador de Telegram	`orchestrator/notify/`
Guía de la demostración del hito	`docs/sprints/sprint2-demostracion.md` (#177)
Planificación detallada del sprint 3	`docs/sprints/sprint3-planificacion.md` (#177)

Cómo reproducir la base de conocimiento y las decisiones en local

Con Neo4j en marcha (`docker start fw-neo4j`) y OPA instalado, desde la raíz del repositorio:

```
uv run framework conocimiento validar
uv run framework conocimiento embeddings
uv run framework conocimiento consultar "guardar contraseñas con hash MD5"
opa test policies/ -v
uv run framework gates decidir --gate 1 --salida tests/datos/gates --compilacion-ok
--no-bloquear
uv run framework gates listar
```